

Using pixi environments for orgmode notebooks

Rohit Goswami · 2026-01-29 · pixi · orgmode · workflow

Shell

To ensure shell blocks recognize project-specific paths and variables, use a named session and initialize the environment once. This allows subsequent blocks to inherit the state without repetitive boilerplate.

```
#+PROPERTY: header-args: bash :session vizSH :results silent
```

Initialize the environment within the session:

```
export GITROOT=$(git rev-parse --show-toplevel)
pixi shell -e eon
```

Python

Basically the executable must be set to the right environment `:python (string-trim (shell-command-to-string "pixi run \"which python\""))`. This means a working setup for a per-file notebook can be:

```
+PROPERTY: header-args: python :session onePy
+PROPERTY: header-args: python :results output :exports both
+PROPERTY: header-args: python :cache yes :tangle pylog.py
+PROPERTY: header-args: python :python "pixi run -e eon python"
```

This also handles `subprocess` calls without monkeying around with `prologue`.

R

For `R`, the integration via `ess` (Emacs Speaks Statistics) requires a bit more care to ensure the interactive session and the babel execution engine stay in sync. Setting the local path and updating the `exec-path` ensures that library lookups don't leak into the system R installation.

```
(let* ((root (vc-root-dir))
      (bin-dir (expand-file-name ".pixi/envs/rviz/bin" root))
      (r-path (expand-file-name "R" bin-dir)))
  (setq-local inferior-ess-r-program r-path)
  (setq-local org-babel-R-command (concat r-path " --slave --no-save"))
  (setq-local exec-path (cons bin-dir exec-path))
  (setenv "PATH" (concat bin-dir ":" (getenv "PATH"))))
(setq-local ess-r-executable-list (list r-path)))
```